KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Project
Football match predictor with machine learning
Geavanceerde computerarchitectuur

Wouter Fouquet
Florian Vandenbogaerde
Master elektronica ICT
Academiejaar 2024-2025

CONTENTS

1 Models **2**
1.1 Logistic Regression (2 Classes: Sigmoid) 2
1.1.1 The Sigmoid Function 2
1.1.2 Suitability for GPU/CPU Benchmarking 3
1.1.3 Gradient Descent and Hyperparameters 3
1.2 K-Nearest Neighbours 3
1.2.1 Parallelizability and Suitability for GPU/CPU Comparison 4

2 Data preparation **5**

3 Model Training and evaluation **7**
3.1 Logistic regression 7
3.1.1 Flowchart 7
3.1.2 Measurements 8
3.1.3 Performance analysis 8
3.2 K-Nearest Neighbours 9
3.2.1 Flowchart 9
3.2.2 Measurements 9
3.2.3 Performance analysis 9

INTRODUCTION

In this project, we aim to design a machine learning model capable of predicting the outcomes of Premier League football matches. Specifically, our goal is to determine whether the home team will win or not, framing this as a binary classification problem. We had this idea because we are 2 fans of the premier league and we wanted to make something in our intresses.

To achieve this, we focus on using models that are logically suited for this task, including logistic regression and k-nearest neighbors (KNN). We have intentionally avoided models like decision trees, as they build sequentially and do not lend themselves well to parallelization, which we consider a limitation in this context.

The dataset used for this project was sourced from Kaggle and covers match data from 2000 to 2018:

https://www.kaggle.com/code/saife245/football-match-prediction/

To connect this project with the course Advanced Computer Architecture, we evaluate the performance of our models on both CPU and GPU architectures. This allows us to analyze the computational efficiency and scalability of each model, and to determine which setup yields the best performance in terms of training and inference speed.

For this project, we selected two machine learning models: logistic regression and k-nearest neighbors who are particularly suitable for our goal of comparing performance between CPU and GPU architectures.

Parallelizability is a key factor in computational efficiency. By selecting models with clear opportunities for parallel computation, we aim to highlight the performance differences between CPU and GPU execution environments.

Importantly, we implemented these models from scratch in C++, without relying on external machine learning libraries. This approach allows us to gain a deeper understanding of the inner workings of each algorithm and ensures that we have full control over the computational structure, which is essential for fair and accurate performance benchmarking.

1.1 Logistic Regression (2 Classes: Sigmoid)

Logistic regression is a widely used algorithm for binary classification tasks. In the context of our project, it is an appropriate and natural choice for predicting whether the home team will win a football match or not. The model's simplicity, efficiency, and interpretability make it a strong baseline for this kind of prediction problem.

Logistic regression functions as a linear model that predicts the probability of a given class occurring for example, whether the home team wins. The model computes a weighted sum of the input features, which is then passed through a sigmoid function to map the result to a probability between 0 and 1. The training process of logistic regression is iterative: during each iteration (epoch), the weights are adjusted based on how well the model performs on the training data. This is done using gradient descent, an optimization technique aimed at minimizing the cost function, typically the log-loss or cross-entropy.

1.1.1 The Sigmoid Function

At the core of logistic regression is the sigmoid activation function, which maps any real-valued input to a value between 0 and 1. This property makes it suitable for modeling probabilities. The sigmoid function is defined as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In our case, the output of the sigmoid represents the probability that the home team will win the match. A threshold (typically 0.5) is then applied to convert this probability into a binary prediction.

1.1.2 Suitability for GPU/CPU Benchmarking

Training the model involves linear algebra operations such as dot products, matrix multiplications, and element-wise computations. These operations can be executed efficiently in parallel, especially on GPU hardware.

This makes logistic regression particularly well-suited for our comparative analysis between CPU and GPU performance.

1.1.3 Gradient Descent and Hyperparameters

Gradient descent is an optimization algorithm used to update the weights and bias of the model based on the computed gradients. The goal is to minimize the cost function by adjusting the parameters in the direction of the negative gradient. This ensures that with each update, the model becomes slightly better at making accurate predictions.

In logistic regression, this means that the weights and bias are updated in response to the error between the predicted outputs and the actual labels. The larger the error, the more significant the adjustment. This process is repeated iteratively until the cost function converges or a predefined number of iterations is reached.

Two key hyperparameters play a crucial role in this process:

- **Learning rate**: This controls the step size of each update. A small learning rate may lead to slow convergence, while a learning rate that is too large can cause the model to overshoot the optimal values or even diverge.
- **Epochs**: This refers to the number of complete passes through the training dataset. More epochs allow the model to learn more thoroughly from the data, but too many can lead to overfitting, where the model performs well on training data but poorly on unseen data.

Careful tuning of these hyperparameters is essential to achieve a good balance between model accuracy and generalization.

1.2 K-Nearest Neighbours

The K-Nearest Neighbours (KNN) algorithm is a simple yet powerful non-parametric method used for classification tasks.

KNN classifies a new data point by identifying the k most similar instances from the training dataset and assigning the most common class among them. The similarity between data points is typically measured using a distance metric. In this project, we use the Euclidean distance, which is one of the most common choices for continuous feature spaces.

The Euclidean distance between two feature vectors $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ and $\mathbf{y} = (y_1, y_2, \ldots, y_n)$ is defined as:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

This metric calculates the straight-line distance between two points in n-dimensional space. Its simplicity and efficiency make it well-suited for high-performance implementations on both CPU and GPU architectures.

1.2.1 Parallelizability and Suitability for GPU/CPU Comparison

One of the main computational challenges in KNN lies in the distance calculation step. For each prediction, the algorithm computes the distance between the test sample and every point in the training dataset.

Despite being computationally intensive, KNN is highly parallelizable. Distance computations for each pair of data points are independent, which makes the algorithm well-suited for execution on GPU architectures where thousands of threads can run concurrently. This provides a valuable opportunity to benchmark the performance difference between CPU and GPU execution.

2 DATA PREPARATION

Before training our models, we first need to preprocess the raw match data into a format suitable for machine learning computation. For convenience and flexibility, especially given the availability of powerful data manipulation libraries such as **pandas** and **scikit-learn**, we perform this preprocessing step in Python. The resulting cleaned dataset is then exported and used for model training and testing in `cu` (C++ with CUDA), which allows us to benchmark CPU and GPU performance. Table 2.1 shows the original table.

Abbreviation	Explanation	Datatype	Useful?
Index		int64	No
Date		Object	No
Home team		Object	No
Away team		Object	No
FTHG	Full Time Home Team Goals	int64	No
FTAG	Full Time Away Team Goals	int64	No
FTR	Full Time Result (H = Home win, NH = Not Home Win)	Object	Target
HTGS	Home Team Goals Scored	int64	No
ATGS	Away Team Goals Scored	int64	No
HTGC	Home Team Goals Conceded	int64	No
ATGC	Away Team Goals Conceded	int64	No
HTP	Home Team Average Points	float64	Yes
ATP	Away Team Average Points	float64	Yes
HM1–HM5	Home last 5 results (M = Undefined, H = Win, D = Draw, A = Loss)	Object	Yes
AM1–AM5	Away last 5 results (M = Undefined, H = Win, D = Draw, A = Loss)	Object	Yes
MW	Match Week	float64	No
HTFormPtsStr	Home Team Form string (M, W, D, L) from HM1–5	Object	No
ATFormPtsStr	Away Team Form string (M, W, D, L) from AM1–5	Object	No
HTFormPts	Home Team Form Points from HTFormPtsStr	int64	No
ATFormPts	Away Team Form Points from ATFormPtsStr	int64	No
HTWinStreak3	Home Win Streak of 3 (1=true, 0=false)	int64	No
HTWinStreak5	Home Win Streak of 5 (1=true, 0=false)	int64	No
HTLossStreak3	Home Lose Streak of 3 (1=true, 0=false)	int64	No
HTLossStreak5	Home Lose Streak of 5 (1=true, 0=false)	int64	No
ATWinStreak3	Away Win Streak of 3 (1=true, 0=false)	int64	No
ATWinStreak5	Away Win Streak of 5 (1=true, 0=false)	int64	No
ATLossStreak3	Away Lose Streak of 3 (1=true, 0=false)	int64	No
ATLossStreak5	Away Lose Streak of 5 (1=true, 0=false)	int64	No
HTGD	Home Team Goal Difference	float64	Yes
ATGD	Away Team Goal Difference	float64	Yes
DiffPts	Points Difference	float64	Yes
DiffFormPts	Form Points Difference	float64	Yes

Table 2.1: Dataset features and metadata

Although the original dataset contains a large number of features, not all of them contribute meaningfully to the predictive performance of our models. Some features, such as team names or match dates, carry no numerical or structural information relevant for classification. Others are redundant because they are used to derive more informative composite features.

To improve model efficiency and focus on the most relevant inputs, we reduce the dataset to a subset of carefully selected features. The resulting reduced feature set is shown in Table 2.2.

Abbreviation	Explanation	Datatype
FTR	Full Time Result (H = Home win, NH = Not Home Win)	Object
HTP	Home Team Average Points	float64
ATP	Away Team Average Points	float64
HM1–HM5	Home last 5 results (M = Undefined, H = Win, D = Draw, A = Loss)	Object
AM1–AM5	Away last 5 results (M = Undefined, H = Win, D = Draw, A = Loss)	Object
HTGD	Home Team Goal Difference	float64
ATGD	Away Team Goal Difference	float64
DiffFormPts	Form Points Difference	float64

Table 2.2: Reduced features

Several of the selected features—particularly those representing the recent match results (HM1 to AM3), are categorical in nature. To make these features usable in our models, we apply one-hot encoding to convert the categorical values into a numerical format. This process transforms each unique category into a binary vector, enabling the model to interpret and learn from these previously non-numeric inputs. The resulting encoded dataset is shown in Table 2.3.

Abbreviation	Explanation	Datatype
FTR	Full Time Result (H = Home win, NH = Not Home Win)	Object
HTP	Home Team Average Points	float64
ATP	Away Team Average Points	float64
HM1_D	Home last result (M = Undefined, H = Win, D = Draw, A = Loss)	Object
HM1_L	Home last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM1_W	Home last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM2_D	Home second to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM2_L	Home second to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM2_W	Home second to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM3_D	Home third to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM3_L	Home third to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HM3_W	Home third to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM1_D	Away last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM1_L	Away last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM1_M	Away last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM2_D	Away second to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM2_L	Away second to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM2_M	Away second to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM3_D	Away third to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM3_L	Away third to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
AM3_M	Away third to last result (M = Undefined, H = Win, D = Draw, A = Loss)	Bool
HTGD	Home Team Goal Difference	float64
ATGD	Away Team Goal Difference	float64
DiffFormPts	Form Points Difference	float64

Table 2.3: Encoded features

This final, cleaned and encoded dataset is then exported and used as input for our custom implementations in `cu`. By doing so, we are able to leverage the GPU for high-performance training and inference.

In this section, both models are trained and evaluated. To assess performance, we implemented four functionally similar programs in CUDA, each corresponding to a specific combination of model type and hardware:

- Logistic Regression on the CPU
- Logistic Regression on the GPU
- K-Nearest Neighbours on the CPU
- K-Nearest Neighbours on the GPU

Initially, we conducted tests using a realistic dataset of approximately 6,800 samples. This allowed us to evaluate both performance and classification accuracy under practical conditions.

To better examine the performance differences between CPU and GPU execution, we then expanded the dataset by introducing dummy data, bringing the total to 100,000 samples. This large-scale simulation emphasizes runtime behavior and scalability. Note that classification accuracy is no longer meaningful in this context due to the artificial nature of the added data.

As with most models, the implementation begins by loading the dataset from a CSV file. Next, the data is split into training and test sets to allow proper model evaluation. After this, the features and corresponding labels are separated to form the input (X) and output (y) vectors used during training.

For GPU-based execution, memory allocation is performed using CUDA constructs, ensuring that all necessary arrays and model parameters are transferred to the device.

3.1 Logistic regression

3.1.1 Flowchart

The steps below outline the general workflow for implementing and executing a logistic regression model on the GPU.

The workflow for the CPU is identical, except that explicit memory allocation is not required. The corresponding code can be found in Appendix 3.1 and 3.2.

- **Read the dataset**
- **Allocate GPU memory**
- **Split the dataset**

- **Separate features and targets**
- **Train the model**:
  - Iterate over a fixed number of epochs.
  - In each epoch, compute predictions and update weights and bias
- **Retrieve model parameters**: Retrieve the trained weights and bias
- **Evaluate the model**: Use the learned parameters on the test samples
- **Clean up memory**

3.1.2 Measurements

Table 3.1 shows the performance results of the logistic regression model on a dataset of 6,800 real samples, using various epoch settings. Table 3.2 presents results on a much larger, synthetic dataset containing 100,000 samples, created to benchmark scalability.

Learning Rate	Epochs	CPU [s]	GPU [s]
0.1	1,000	1.057	0.360
0.1	10,000	10.374	3.199
0.1	100,000	110.470	31.426

Table 3.1: Logistic Regression performance on 6,800 real samples

Learning Rate	Epochs	CPU [s]	GPU [s]
0.1	1,000	29.067	7.471
0.1	10,000	384.437	74.826

Table 3.2: Logistic Regression performance on 100,000 synthetic samples

3.1.3 Performance analysis

The results for logistic regression clearly show that GPU acceleration offers a consistent performance benefit across different dataset sizes and numbers of training epochs. For the smaller dataset of 6,800 real samples, the GPU implementation is approximately 3 times faster than the CPU, reducing execution time from 1.06 to 0.36 seconds at 1,000 epochs, and from 110.47 to 31.43 seconds at 100,000 epochs. This demonstrates that as the computational load increases, the performance gap between CPU and GPU grows wider in favor of the GPU.

This trend is even more pronounced for the larger dataset of 100,000 synthetic samples. At 10,000 epochs, the GPU completes training in approximately 75 seconds, whereas the CPU requires nearly 385 seconds — a speedup factor of over 5x.

It is important to note that the learning rate has no significant impact on the timing measurements, indicating that the observed performance differences are primarily driven by dataset size and training duration rather than hyperparameter settings.

Overall, these measurements highlight that logistic regression benefits significantly from parallel execution on the GPU, particularly in scenarios involving large datasets and extended training durations.

3.2 K-Nearest Neighbours

3.2.1 Flowchart

The steps below outline the general workflow for implementing and executing a K-Nearest Neighbors (KNN) classifier on the GPU.

The workflow for the CPU is identical, except that memory allocation is not required. The corresponding code can be found in Appendix 3.3 and 3.4.

- **Read the dataset**
- **Allocate GPU memory**
- **Split the dataset**
- **Separate features and targets**
- **Select hyperparameter *k***: Choose the number of nearest neighbours to consider for classification.
- **Perform inference**:
  - Compute the Euclidean distance between each test sample and all training samples.
  - Sort the distances to identify the k closest training instances.
  - Assign the predicted label based on a majority vote among the k nearest neighbours.
- **Evaluate the model**
- **Clean up memory**

3.2.2 Measurements

The performance of the KNN algorithm is presented in Table 3.3 for the real dataset with 6,800 samples and in Table 3.4 for the synthetic dataset with 100,000 samples.

k	CPU [s]	GPU [s]
3	3.853	1.232
10	3.725	1.214

Table 3.3: K-Nearest Neighbours performance on 6,800 real samples

k	CPU [s]	GPU [s]
3	906.670	8.047
10	983.655	9.143

Table 3.4: K-Nearest Neighbours performance on 100,000 synthetic samples

3.2.3 Performance analysis

The performance results clearly demonstrate the advantage of GPU acceleration for the K-Nearest Neighbours (KNN) algorithm, particularly as the dataset size increases.

For the smaller dataset of 6,800 real samples, the GPU implementation achieves a speedup of approximately 3 times compared to the CPU version, reducing execution time from around 3.8 seconds to just over 1.2 seconds.

This speedup becomes significantly more pronounced on the larger dataset of 100,000 synthetic samples. Here, the CPU requires over 900 seconds to complete the task, whereas the GPU performs the same operation in under 10 seconds, achieving a speedup factor of more than 100x.

These results highlight the scalability and efficiency benefits of parallel computation with CUDA when working with large-scale KNN classification tasks.

In this project, we implemented and benchmarked two machine learning models, Logistic Regression and K-Nearest Neighbours on both CPU and GPU architectures. The primary objective was to assess whether GPU execution offers a significant performance advantage over CPU execution in practical scenarios.

Based on the performance measurements, we conclude that GPU execution is indeed significantly faster than CPU execution, especially for large datasets. While the benefits of GPU acceleration are less prominent on smaller datasets (such as the one with 6,800 real samples), they become substantial when scaling to 100,000 samples. This confirms the suitability of GPU architectures for highly parallelizable tasks, such as matrix operations in Logistic Regression and distance calculations in KNN.

However, it is important to note that the GPU advantage only becomes clear when there is sufficient computational load. On small datasets with relatively few features, the overhead of transferring data to and from the GPU may diminish or even outweigh the performance gains. Therefore, the effectiveness of GPU acceleration depends not only on the algorithm but also on the dataset size and feature richness.

Recommendations and Future Improvements

Although our implementation successfully demonstrates the benefits of GPU computation, there are several opportunities for further optimization and improvement:

- **Shared memory usage:** Currently, all memory accesses are performed via global memory, which is relatively slow. In models like KNN, where features are repeatedly accessed during distance computation, using shared memory can significantly reduce memory latency and improve throughput.
- **Kernel optimization:** Further tuning of CUDA kernels, including thread/block configuration, memory coalescing, and minimizing divergence, can yield better performance.
- **Data pipeline optimization:** Currently, data preprocessing is performed in Python. By integrating more preprocessing steps directly into our own C++/CUDA pipeline, we can eliminate the need for external Python scripts and further streamline the workflow.
- **Model complexity:** Adding more informative features or using engineered features could improve model performance and allow for more realistic performance benchmarking under practical workloads.

These improvements would allow for more efficient use of the GPU's capabilities and make the results even more representative of real-world machine learning systems.

Listing 3.1: Logistic Regression on CPU

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <stdio.h>
#include <limits>
#include <cmath>
#include <chrono>


using namespace std;
using namespace std::chrono;

vector<vector<float>> read_csv(const string& filename) {
    ifstream file(filename);
    vector<vector<float>> data;
    string line;

    if (!file.is_open()) {
        cerr << "Cannot open file: " << filename << endl;
        return {};
    }

    getline(file, line); // Skip header

    int line_num = 1;
    while (getline(file, line)) {
        line_num++;
        stringstream ss(line);
        vector<float> row;
        string value;

        while (getline(ss, value, ',')) {
            try {
                if (value == "True") {
                    row.push_back(1.0f);
                } else if (value == "False") {
                    row.push_back(0.0f);
                } else {
                    row.push_back(stof(value));  // Try float conversion
                }
            } catch (const exception& e) {
                cerr << "Line " << line_num << ": Invalid value '" << value
                     << "'" << endl;
                row.clear();  // skip the whole row on error
                break;
            }
        }

        if (!row.empty())
            data.push_back(row);
    }

    return data;
}


void print_first_five_rows(const vector<vector<float>>& data) {
    cout << "Eerste 5 rijen:" << endl;
```

```
60      for (size_t i = 0; i < min(data.size(), size_t(5)); ++i) {
61          for (float v : data[i]) {
62              cout << v << " ";
63          }
64          cout << endl;
65      }
66  }
67
68
69  void split_data(const vector<vector<float>>& data, vector<vector<float>>&
        train_data, vector<vector<float>>& test_data, float test_size) {
70      size_t total_size = data.size();
71      size_t test_size_count = static_cast<size_t>(total_size * test_size);
72      size_t train_size_count = total_size - test_size_count;
73
74      train_data.assign(data.begin(), data.begin() + train_size_count);
75      test_data.assign(data.begin() + train_size_count, data.end());
76  }
77
78
79  void split_features_targets(const vector<vector<float>>& dataset, vector<
        vector<float>>& features, vector<float>& targets) {
80      for (const auto& row : dataset) {
81          targets.push_back(row[0]);                              // FTR as
                target
82          features.emplace_back(row.begin() + 1, row.end());  // other
                columns are features
83      }
84  }
85
86
87  float calculate_accuracy(const float* predictions, const float* targets,
        int num_samples) {
88      int correct = 0;
89      for (int i = 0; i < num_samples; ++i) {
90          int predicted_label = predictions[i] >= 0.5f ? 1 : 0;
91          int true_label = static_cast<int>(targets[i]);
92          if (predicted_label == true_label) {
93              ++correct;
94          }
95      }
96      return static_cast<float>(correct) / num_samples;
97  }
98
99
100 void printModelParameters(float* weights, float bias, int num_features) {
101     cout << "\n--- Trained Model Parameters ---" << endl;
102     cout << "Weights:\n";
103     for (int j = 0; j < num_features; ++j) {
104         cout << "  w[" << j << "] = " <<  weights[j] << endl;
105     }
106     cout << "Bias:\n  b = " << bias << endl;
107 }
108
109
110 float sigmoidCPU(float z) { // Sigmoid function is used to map the output
        of the neuron to a probability between 0 and 1
111     return 1.0f / (1.0f + expf(-z));
112 }
113
114
```

```
115 //predict_kernel is a kernel that calculates the predictions of the model
        based on the input features, weights and bias. It uses the sigmoid
        function to map the output of the neuron to a probability between 0 and
         1.
116 void predict_cpu(const float* features, const float* weights, const float*
        bias, float* predictions, int num_samples, int num_features) {
117     for (int i = 0; i < num_samples; ++i) {
118         float z = 0.0f;
119         for (int j = 0; j < num_features; ++j) {
120             z += features[i * num_features + j] * weights[j];
121         }
122         z += *bias;
123         predictions[i] = sigmoidCPU(z);
124     }
125 }
126
127
128 void update_weights_cpu(const float* features, const float* targets, const
        float* predictions, float* weights, float* bias, float lr, int
        num_samples, int num_features) {
129     for (int i = 0; i < num_features; ++i) {
130         float grad_w = 0.0f;
131         for (int j = 0; j < num_samples; ++j) {
132             grad_w += (predictions[j] - targets[j]) * features[j *
                    num_features + i];
133         }
134         weights[i] -= lr * grad_w / num_samples;
135     }
136
137     float grad_b = 0.0f;
138     for (int j = 0; j < num_samples; ++j) {
139         grad_b += predictions[j] - targets[j];
140     }
141     *bias -= lr * grad_b / num_samples;
142 }
143
144
145 int main() {
146     string filename = "data_clean_expanded_100000.csv";
147     vector<vector<float>> data = read_csv(filename);
148
149
150     //print_first_five_rows(data);
151
152     //--------------------
153     // GENERAL SETUP
154     //--------------------
155
156     int num_features = data[0].size() - 1; // number of features (excluding
             the target column)
157     cout << "Aantal␣features:␣" << num_features << endl;
158
159     vector<float> weights(num_features, 0.0f);
160     float bias = 0.0f;
161
162
163     // -------------------
164     // DATA SPLITTING
165     // -------------------
166
167     vector<vector<float>> train_data, test_data;
```

```
168     float test_size = 0.2f; // 20% of the data for testing
169     split_data(data, train_data, test_data, test_size);
170
171
172     vector<vector<float>> train_features_vec, test_features_vec;
173     vector<float> train_targets_vec, test_targets_vec;
174     split_features_targets(train_data, train_features_vec,
            train_targets_vec);
175     split_features_targets(test_data, test_features_vec, test_targets_vec);
176
177
178     // --------------------
179     // TRAINING SETUP
180     // --------------------
181
182     auto start_cpu = high_resolution_clock::now();
183
184
185     int num_train_samples = train_features_vec.size();
186
187     vector<float> train_features_flat(num_train_samples * num_features);
188     // Flatten the 2D vector to a 1D array for pointer-based functions for
            better comparison with CUDA
189     for (int i = 0; i < num_train_samples; ++i){
190         copy(train_features_vec[i].begin(), train_features_vec[i].end(), &
                train_features_flat[i * num_features]);
191     }
192
193     const float* train_features = train_features_flat.data();
194     const float* train_targets = train_targets_vec.data();
195     vector<float> train_predictions(num_train_samples);
196
197     // --------------------
198     // TRAINING
199     // --------------------
200
201     float lr = 0.1f;
202     int epochs = 10000;
203
204     cout << "Aantal epochs: " << epochs << endl;
205     cout << "Learning rate: " << lr << endl;
206
207     // Training loop
208     for (int epoch = 0; epoch < epochs; ++epoch) {
209         predict_cpu(train_features, weights.data(), &bias,
                train_predictions.data(), num_train_samples, num_features);
210         update_weights_cpu(train_features, train_targets, train_predictions
                .data(), weights.data(), &bias, lr, num_train_samples,
                num_features);
211     }
212
213
214     // --------------------
215     // MODEL PARAMETERS
216     // --------------------
217
218     // not used in this version
219     //printModelParameters(weights.data(), bias, num_features);
220
221
222     // --------------------
```

```
223     // TESTING SETUP
224     // --------------------
225
226
227
228     int num_test_samples = test_features_vec.size();
229     cout << "Aantal test samples: " << num_test_samples << endl;
230
231     vector<float> test_features_flat(num_test_samples * num_features);
232     for (int i = 0; i < num_test_samples; ++i){
233         copy(test_features_vec[i].begin(), test_features_vec[i].end(), &
                test_features_flat[i * num_features]);
234     }
235
236     const float* test_features = test_features_flat.data();
237     const float* test_targets = test_targets_vec.data();
238     vector<float> test_predictions(num_test_samples);
239
240
241     // --------------------
242     // TESTING
243     // --------------------
244
245     predict_cpu(test_features, weights.data(), &bias, test_predictions.data
            (), num_test_samples, num_features);
246
247
248     auto stop_cpu = high_resolution_clock::now();
249     auto duration_cpu = duration_cast<milliseconds>(stop_cpu - start_cpu);
250     cout << "CPU Logistic Regression Execution Time: " << duration_cpu.
            count() << " ms." << endl;
251
252     float acc = calculate_accuracy(test_predictions.data(), test_targets,
            num_test_samples);
253     cout << "\nTest Accuracy: " << acc * 100.0f << "%" << endl;
254
255     return 0;
256 }
```

Listing 3.2: Logistic Regression on GPU

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <stdio.h>
#include <limits>
#include <cmath>
#include <chrono>



using namespace std;
using namespace std::chrono;

vector<vector<float>> read_csv(const string& filename) {
    ifstream file(filename);
    vector<vector<float>> data;
    string line;

    if (!file.is_open()) {
        cerr << "Cannot open file: " << filename << endl;
        return {};
    }

    getline(file, line); // Skip header

    int line_num = 1;
    while (getline(file, line)) {
        line_num++;
        stringstream ss(line);
        vector<float> row;
        string value;

        while (getline(ss, value, ',')) {
            try {
                if (value == "True") {
                    row.push_back(1.0f);
                } else if (value == "False") {
                    row.push_back(0.0f);
                } else {
                    row.push_back(stof(value));  // Try float conversion
                }
            } catch (const exception& e) {
                cerr << "Line " << line_num << ": Invalid value '" << value
                     << "'" << endl;
                row.clear();  // skip the whole row on error
                break;
            }
        }

        if (!row.empty())
            data.push_back(row);
    }

    return data;
}


void print_first_five_rows(const vector<vector<float>>& data) {
```

```
60      cout << "Eerste 5 rijen:" << endl;
61      for (size_t i = 0; i < min(data.size(), size_t(5)); ++i) {
62          for (float v : data[i]) {
63              cout << v << " ";
64          }
65          cout << endl;
66      }
67  }
68
69
70  void split_data(const vector<vector<float>>& data, vector<vector<float>>&
        train_data, vector<vector<float>>& test_data, float test_size) {
71      size_t total_size = data.size();
72      size_t test_size_count = static_cast<size_t>(total_size * test_size);
73      size_t train_size_count = total_size - test_size_count;
74
75      train_data.assign(data.begin(), data.begin() + train_size_count);
76      test_data.assign(data.begin() + train_size_count, data.end());
77  }
78
79
80  void split_features_targets(const vector<vector<float>>& dataset, vector<
        vector<float>>& features, vector<float>& targets) {
81      for (const auto& row : dataset) {
82          targets.push_back(row[0]);                          // FTR as target
83          features.emplace_back(row.begin() + 1, row.end()); // other columns
                are features
84      }
85  }
86
87
88  float calculate_accuracy(const float* predictions, const float* targets,
        int num_samples) {
89      int correct = 0;
90      for (int i = 0; i < num_samples; ++i) {
91          int predicted_label = predictions[i] >= 0.5f ? 1 : 0;
92          int true_label = static_cast<int>(targets[i]);
93          if (predicted_label == true_label) {
94              ++correct;
95          }
96      }
97      return static_cast<float>(correct) / num_samples;
98  }
99
100
101 void printModelParameters(float* weights, float bias, int num_features) {
102     cout << "\n--- Trained Model Parameters ---" << endl;
103     cout << "Weights:\n";
104     for (int j = 0; j < num_features; ++j) {
105         cout << "  w[" << j << "] = " <<  weights[j] << endl;
106     }
107     cout << "Bias:\n  b = " << bias << endl;
108 }
109
110
111 //------------------------------GPU KERNELS------------------------------//
112
113
114 __device__ float sigmoidGPU(float z) { // Sigmoid function is used to map
        the output of the neuron to a probability between 0 and 1
115     return 1.0f / (1.0f + expf(-z));
```

```
116 }
117
118
119 //predict_kernel is a kernel that calculates the predictions of the model
        based on the input features, weights and bias. It uses the sigmoid
        function to map the output of the neuron to a probability between 0 and
         1
120 __global__ void predict_kernel(float* features, float* weights, float* bias
        , float* predictions, int num_samples, int num_features) {
121     int i = blockIdx.x * blockDim.x + threadIdx.x;
122     if (i >= num_samples) return;
123
124     float z = 0.0f;
125     for (int j = 0; j < num_features; ++j){
126         z += features[i * num_features + j] * weights[j];
127     }
128     z += *bias;
129     predictions[i] = sigmoidGPU(z);
130 }
131
132
133 //update_weights_kernel is a kernel that updates the weights and bias of
        the model based on the predictions and targets. It uses the gradient
        descent algorithm to update the weights and bias
134 __global__ void update_weights_kernel(float* features, float* targets,
        float* predictions, float* weights, float* bias, float lr, int
        num_samples, int num_features) {
135     int i = threadIdx.x;
136     if (i >= num_features) return;
137
138     float grad_w = 0.0f;                            //gradient of the weights
139     for (int j = 0; j < num_samples; ++j){
140         grad_w += (predictions[j] - targets[j]) * features[j * num_features
                + i];
141     }
142
143     weights[i] -= lr * grad_w / num_samples;
144
145     if (i == 0) {
146         float grad_b = 0.0f;
147         for (int j = 0; j < num_samples; ++j){
148             grad_b += predictions[j] - targets[j];
149         }
150     *bias -= lr * grad_b / num_samples;
151     }
152 }
153
154
155
156 int main() {
157     cout << "Reading␣CSV␣file..." << endl;
158     string filename = "data_clean_expanded_100000.csv";
159     vector<vector<float>> data = read_csv(filename);
160
161     //print_first_five_rows(data);
162
163
164     //--------------------
165     // GENERAL SETUP
166     //--------------------
167
```

```
168     int num_features = data[0].size() - 1; // number of features (columns)
            in the dataset
169     cout << "Aantal features: " << num_features << endl;
170
171     float* d_weights, *d_bias;
172
173     cudaMalloc(&d_weights, num_features * sizeof(float));
174     cudaMalloc(&d_bias, sizeof(float));
175
176
177     // --------------------
178     // DATA SPLITTING
179     // --------------------
180     vector<vector<float>> train_data, test_data;
181     float test_size = 0.2f; // 20% of the data for testing
182     split_data(data, train_data, test_data, test_size);
183
184     vector<vector<float>> train_features, test_features;
185     vector<float> train_targets, test_targets;
186     split_features_targets(train_data, train_features, train_targets);
187     split_features_targets(test_data, test_features, test_targets);
188
189     // --------------------
190     // TRAINING SETUP
191     // --------------------
192
193     auto start_GPU = high_resolution_clock::now();
194
195     int num_train_samples = train_features.size();
196     cout << "Aantal train samples: " << num_train_samples << endl;
197
198     float* h_train_features = new float[num_train_samples * num_features];
199     float* h_train_targets = new float[num_train_samples];
200
201     //copy the features and targets from the dataset to the h_features and
            h_targets pointers
202     for (int i = 0; i < num_train_samples; ++i) {
203         for (int j = 0; j < num_features; ++j){
204             h_train_features[i * num_features + j] = train_features[i][j];
205         }
206         h_train_targets[i] = train_targets[i];
207     }
208
209     float* d_train_features, *d_train_targets, *d_train_predictions;
210
211
212     cudaMalloc(&d_train_features, num_train_samples * num_features * sizeof
            (float));
213     cudaMalloc(&d_train_targets, num_train_samples * sizeof(float));
214
215     cudaMalloc(&d_train_predictions, num_train_samples * sizeof(float));
216
217     cudaMemcpy(d_train_features, h_train_features, num_train_samples *
            num_features * sizeof(float), cudaMemcpyHostToDevice);
218     cudaMemcpy(d_train_targets, h_train_targets, num_train_samples * sizeof
            (float), cudaMemcpyHostToDevice);
219     cudaMemset(d_weights, 0, num_features * sizeof(float));     //
            Initialise weights with 0
220     cudaMemset(d_bias, 0, sizeof(float));                       //
            Initialise bias with 0
221
```

```
222
223     // --------------------
224     // TRAINING
225     // --------------------
226     float lr = 0.01f;   // Learning rate
227     int epochs = 10000;  // Number of epochs
228
229     cout << "Aantal epochs: " << epochs << endl;
230     cout << "Learning rate: " << lr << endl;
231
232     // Training loop
233     for (int epoch = 0; epoch < epochs; ++epoch) {
234         predict_kernel<<<(num_train_samples + 255) / 256, 256>>>(
                d_train_features, d_weights, d_bias, d_train_predictions,
                num_train_samples, num_features);
235         update_weights_kernel<<<1, num_features>>>(d_train_features,
                d_train_targets, d_train_predictions, d_weights, d_bias, lr,
                num_train_samples, num_features);
236     }
237
238
239     // Training loop with logging
240     /*
241     for (int epoch = 0; epoch < epochs; ++epoch) {
242         predict_kernel<<<(num_train_samples + 255) / 256, 256>>>(
                d_train_features, d_weights, d_bias, d_predictions,
                num_train_samples, num_features);
243         update_weights_kernel<<<1, num_features>>>(d_train_features,
                d_train_targets, d_predictions, d_weights, d_bias, lr,
                num_train_samples, num_features);
244
245         // Logging elke 100 epochs
246         if ((epoch + 1) % 100 == 0 || epoch == epochs - 1) {
247             cudaDeviceSynchronize();  // Zorg dat alles klaar is
248             float* h_predictions = new float[num_train_samples];
249             float* h_targets = new float[num_train_samples];
250             cudaMemcpy(h_predictions, d_predictions, num_train_samples *
                    sizeof(float), cudaMemcpyDeviceToHost);
251             cudaMemcpy(h_targets, d_train_targets, num_train_samples *
                    sizeof(float), cudaMemcpyDeviceToHost);
252
253             float train_accuracy = calculate_accuracy(h_predictions,
                    h_targets, num_train_samples);
254             cout << "Epoch " << epoch + 1 << " - Train accuracy: " <<
                    train_accuracy * 100 << "%" << endl;
255
256             delete[] h_predictions;
257             delete[] h_targets;
258         }
259     }
260     */
261
262
263     cudaDeviceSynchronize();
264
265
266     // --------------------
267     // MODEL PARAMETERS
268     // --------------------
269
270     float* h_weights = new float[num_features];
```

```
271     float h_bias;
272     cudaMemcpy(h_weights, d_weights, num_features * sizeof(float),
            cudaMemcpyDeviceToHost);
273     cudaMemcpy(&h_bias, d_bias, sizeof(float), cudaMemcpyDeviceToHost);
274
275
276     //printModelParameters(h_weights, h_bias, num_features);
277
278
279
280     // --------------------
281     // TESTING SETUP
282     // --------------------
283
284     int num_test_samples = test_features.size();
285
286     cout << "Aantal test samples: " << num_test_samples << endl;
287
288     float* h_test_features = new float[num_test_samples * num_features];
289     float* h_test_targets = new float[num_test_samples];
290     float* h_test_predictions = new float[num_test_samples];
291
292     //copy the features and targets from the dataset to the h_features and
            h_targets pointers
293     for (int i = 0; i < num_test_samples; ++i) {
294         for (int j = 0; j < num_features; ++j){
295             h_test_features[i * num_features + j] = test_features[i][j];
296         }
297         h_test_targets[i] = test_targets[i];
298     }
299
300
301     float* d_test_features, *d_test_targets, *d_test_predictions;
302
303     cudaMalloc(&d_test_features, num_test_samples * num_features * sizeof(
            float));
304     cudaMalloc(&d_test_targets, num_test_samples * sizeof(float));
305     cudaMalloc(&d_test_predictions, num_test_samples * sizeof(float));
306
307
308     cudaMemcpy(d_test_features, h_test_features, num_test_samples *
            num_features * sizeof(float), cudaMemcpyHostToDevice);
309     cudaMemcpy(d_test_targets, h_test_targets, num_test_samples * sizeof(
            float), cudaMemcpyHostToDevice);
310
311
312     // --------------------
313     // TESTING
314     // --------------------
315
316     predict_kernel<<<(num_test_samples + 255) / 256, 256>>>(d_test_features
            , d_weights, d_bias, d_test_predictions, num_test_samples,
            num_features);
317     cudaDeviceSynchronize();
318
319
320     cudaMemcpy(h_test_predictions, d_test_predictions, num_test_samples *
            sizeof(float), cudaMemcpyDeviceToHost);
321     cudaMemcpy(h_test_targets, d_test_targets, num_test_samples * sizeof(
            float), cudaMemcpyDeviceToHost);
322
```

```
323
324     auto stop_GPU = high_resolution_clock::now();
325     auto duration_GPU = duration_cast<milliseconds>(stop_GPU - start_GPU);
326     cout << "GPU Logistic Regression Execution Time: " << duration_GPU.
            count() << " ms" << endl;
327
328
329     float accuracy = calculate_accuracy(h_test_predictions, h_test_targets,
             num_test_samples);
330     cout << "Test accuracy: " << accuracy * 100.0f << "%" << endl;
331
332     // Free device memory
333     cudaFree(d_train_features);
334     cudaFree(d_train_targets);
335     cudaFree(d_weights);
336     cudaFree(d_bias);
337     cudaFree(d_train_predictions);
338     cudaFree(d_test_features);
339     cudaFree(d_test_targets);
340     cudaFree(d_test_predictions);
341     delete[] h_test_predictions;
342     delete[] h_train_features;
343     delete[] h_train_targets;
344     delete[] h_test_targets;
345     delete[] h_weights;
346     cudaDeviceReset(); // Reset de GPU om geheugenlekken te voorkomen
347
348     return 0;
349 }
```

Listing 3.3: KNN on CPU

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cmath>
#include <limits>
#include <algorithm>
#include <chrono>

using namespace std;
using namespace std::chrono;


vector<vector<float>> read_csv(const string& filename) {
    ifstream file(filename);
    vector<vector<float>> data;
    string line;

    if (!file.is_open()) {
        cerr << "Cannot open file: " << filename << endl;
        return {};
    }

    getline(file, line); // Skip header

    int line_num = 1;
    while (getline(file, line)) {
        line_num++;
        stringstream ss(line);
        vector<float> row;
        string value;

        while (getline(ss, value, ',')) {
            try {
                if (value == "True") {
                    row.push_back(1.0f);
                } else if (value == "False") {
                    row.push_back(0.0f);
                } else {
                    row.push_back(stof(value));  // Try float conversion
                }
            } catch (const exception& e) {
                cerr << "Line " << line_num << ": Invalid value '" << value
                     << "'" << endl;
                row.clear();  // skip the whole row on error
                break;
            }
        }

        if (!row.empty())
            data.push_back(row);
    }

    return data;
}


void print_first_five_rows(const vector<vector<float>>& data) {
    cout << "Eerste 5 rijen:" << endl;
```

```
60      for (size_t i = 0; i < min(data.size(), size_t(5)); ++i) {
61          for (float v : data[i]) {
62              cout << v << "␣";
63          }
64          cout << endl;
65      }
66  }
67
68
69  void split_data(const vector<vector<float>>& data, vector<vector<float>>&
        train_data, vector<vector<float>>& test_data, float test_size) {
70      size_t total_size = data.size();
71      size_t test_size_count = static_cast<size_t>(total_size * test_size);
72      size_t train_size_count = total_size - test_size_count;
73
74      train_data.assign(data.begin(), data.begin() + train_size_count); //
            first 80% for training
75      test_data.assign(data.begin() + train_size_count, data.end());    //
            last 20% for testing
76  }
77
78
79  void split_features_targets(const vector<vector<float>>& dataset, vector<
        vector<float>>& features, vector<float>& targets) {
80      for (const auto& row : dataset) {
81          targets.push_back(row[0]);                          // FTR as target
82          features.emplace_back(row.begin() + 1, row.end()); // other columns
                are features
83      }
84  }
85
86
87  float calculate_accuracy(const float* predictions, const float* targets,
        int size) {
88      int correct = 0;
89      for (int i = 0; i < size; ++i) {
90          if (static_cast<int>(predictions[i]) == static_cast<int>(targets[i
                ])) {
91              ++correct;
92          }
93      }
94      return static_cast<float>(correct) / size;
95  }
96
97
98  float euclidean_distance(const vector<float>& point1, const vector<float>&
        point2) {
99      float dist = 0.0f;
100     for (size_t i = 0; i < point1.size(); ++i) {
101         float diff = point1[i] - point2[i];
102         dist += diff * diff;
103     }
104     return sqrt(dist);
105 }
106
107
108 void knn_cpu(const vector<vector<float>>& train_features, const vector<
        float>& train_targets, const vector<vector<float>>& test_features,
        vector<float>& test_predictions, int k) {
109     size_t num_test_samples = test_features.size();
110
```

```
    for (size_t i = 0; i < num_test_samples; ++i) {
        const vector<float>& test_sample = test_features[i];

        vector<pair<float, float>> distances;

        // calculate the distances to all training samples
        for (size_t j = 0; j < train_features.size(); ++j) {
            float dist = euclidean_distance(test_sample, train_features[j])
                ;
            distances.push_back(make_pair(dist, train_targets[j]));
        }

        // Sort the distances
        sort(distances.begin(), distances.end(), [](const pair<float, float
            >& a, const pair<float, float>& b) {
            return a.first < b.first;
        });

        // vote for the k nearest neighbors
        int vote_0 = 0, vote_1 = 0;
        for (int j = 0; j < k; ++j) {
            if (distances[j].second == 0) ++vote_0;
            else if (distances[j].second == 1) ++vote_1;
        }

        // Majority vote (2 classes) if there is a tie, prefer class 1
        test_predictions[i] = (vote_1 > vote_0) ? 1.0f : 0.0f;
    }
}



int main(){
    cout << "Reading␣CSV␣file..." << endl;
    string filename = "data_clean.csv";
    vector<vector<float>> data = read_csv(filename);

    //print_first_five_rows(data);


    //--------------------
    // GENERAL SETUP
    //--------------------

    int num_features = data[0].size() - 1;  // number of features (last
        column is the target)
    cout << "Number␣of␣features:␣" << num_features << endl;


    // -------------------
    // DATA SPLITTING
    // -------------------

    vector<vector<float>> train_data, test_data;
    float test_size = 0.2f; // 20% of the data for testing
    split_data(data, train_data, test_data, test_size);

    vector<vector<float>> train_features, test_features;
    vector<float> train_targets, test_targets;
    split_features_targets(train_data, train_features, train_targets);
    split_features_targets(test_data, test_features, test_targets);
```

```
169
170
171     //--------------------
172     // KNN CPU ALGORITHM
173     //--------------------
174
175     int k = 10; // number of neighbors
176     cout << "Number␣of␣neighbours:␣" << k << endl;
177     size_t num_test_samples = test_features.size();
178     vector<float> cpu_test_predictions(num_test_samples);
179
180
181     auto start_cpu = high_resolution_clock::now();
182
183     knn_cpu(train_features, train_targets, test_features,
            cpu_test_predictions, k);
184
185     auto stop_cpu = high_resolution_clock::now();
186     auto duration_cpu = duration_cast<milliseconds>(stop_cpu - start_cpu);
187     cout << "CPU␣Execution␣Time:␣" << duration_cpu.count() << "␣ms" << endl
            ;
188
189     // Bereken de nauwkeurigheid voor de CPU-versie
190     float cpu_accuracy = calculate_accuracy(cpu_test_predictions.data(),
            test_targets.data(), num_test_samples);
191     cout << "CPU␣KNN␣Accuracy:␣" << cpu_accuracy * 100.0f << "%" << endl;
192
193 }
```

Listing 3.4: KNN on GPU

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cmath>
#include <limits>
#include <algorithm>
#include <chrono>

using namespace std;
using namespace std::chrono;

#define K_MAX 10

vector<vector<float>> read_csv(const string& filename) {
    ifstream file(filename);
    vector<vector<float>> data;
    string line;

    if (!file.is_open()) {
        cerr << "Cannot open file: " << filename << endl;
        return {};
    }

    getline(file, line); // Skip header

    int line_num = 1;
    while (getline(file, line)) {
        line_num++;
        stringstream ss(line);
        vector<float> row;
        string value;

        while (getline(ss, value, ',')) {
            try {
                if (value == "True") {
                    row.push_back(1.0f);
                } else if (value == "False") {
                    row.push_back(0.0f);
                } else {
                    row.push_back(stof(value));  // Try float conversion
                }
            } catch (const exception& e) {
                cerr << "Line " << line_num << ": Invalid value '" << value
                     << "'" << endl;
                row.clear();  // skip the whole row on error
                break;
            }
        }

        if (!row.empty())
            data.push_back(row);
    }

    return data;
}


void print_first_five_rows(const vector<vector<float>>& data) {
```

```
60      cout << "Eerste 5 rijen:" << endl;
61      for (size_t i = 0; i < min(data.size(), size_t(5)); ++i) {
62          for (float v : data[i]) {
63              cout << v << " ";
64          }
65          cout << endl;
66      }
67  }
68
69
70  void split_data(const vector<vector<float>>& data, vector<vector<float>>&
        train_data, vector<vector<float>>& test_data, float test_size) {
71      size_t total_size = data.size();
72      size_t test_size_count = static_cast<size_t>(total_size * test_size);
73      size_t train_size_count = total_size - test_size_count;
74
75      train_data.assign(data.begin(), data.begin() + train_size_count); //
            first 80% for training
76      test_data.assign(data.begin() + train_size_count, data.end());    //
            last 20% for testing
77  }
78
79
80  void split_features_targets(const vector<vector<float>>& dataset, vector<
        vector<float>>& features, vector<float>& targets) {
81      for (const auto& row : dataset) {
82          targets.push_back(row[0]);                          // FTR as target
83          features.emplace_back(row.begin() + 1, row.end()); // other columns
                are features
84      }
85  }
86
87
88  float calculate_accuracy(const float* predictions, const float* targets,
        int size) {
89      int correct = 0;
90      for (int i = 0; i < size; ++i) {
91          if (static_cast<int>(predictions[i]) == static_cast<int>(targets[i
                ])) {
92              ++correct;
93          }
94      }
95      return static_cast<float>(correct) / size;
96  }
97
98
99  //-----------------------------GPU KERNELS-----------------------------//
100
101 __device__ float euclidean_distance(const float* point1, const float*
        point2, int num_features) {
102     float dist = 0.0f;
103     for (int i = 0; i < num_features; ++i) {
104         float diff = point1[i] - point2[i];
105         dist += diff * diff;
106     }
107     return sqrtf(dist);
108 }
109
110 __global__ void knn_kernel(const float* train_features, const float*
        test_features, const float* train_targets, float* test_predictions, int
         num_train_samples, int num_test_samples, int num_features, int k) {
```

```
111     int idx = blockIdx.x * blockDim.x + threadIdx.x;
112     if (idx >= num_test_samples || k > K_MAX || k < 1) return; // make sure
            we don't go out of bounds
113
114
115     const float* test_sample = &test_features[idx * num_features];
116
117     float nearest_distances[K_MAX];
118     int nearest_targets[K_MAX];
119
120     // initialise nearest distances and targets arrays
121     for (int i = 0; i < k; ++i) {
122         nearest_distances[i] = FLT_MAX;
123         nearest_targets[i] = -1;
124     }
125
126     // search for the k nearest neighbors
127     for (int i = 0; i < num_train_samples; ++i) {
128         const float* train_sample = &train_features[i * num_features];
129         float dist = euclidean_distance(test_sample, train_sample,
                num_features);
130
131         // make sure the distance is not already in the nearest distances
132         int max_index = 0;
133         float max_dist = nearest_distances[0];
134         for (int j = 1; j < k; ++j) {
135             if (nearest_distances[j] > max_dist) {
136                 max_dist = nearest_distances[j];
137                 max_index = j;
138             }
139         }
140
141         if (dist < max_dist) {
142             nearest_distances[max_index] = dist;
143             nearest_targets[max_index] = train_targets[i];
144         }
145     }
146
147     // Majority voting (2 classes) if there is a tie, prefer class 1
148     int vote_0 = 0, vote_1 = 0;
149     for (int i = 0; i < k; ++i) {
150         if (nearest_targets[i] == 0) ++vote_0;
151         else if (nearest_targets[i] == 1) ++vote_1;
152     }
153
154     test_predictions[idx] = (vote_1 > vote_0) ? 1.0f : 0.0f;
155 }
156
157
158 int main(){
159     cout << "Reading␣CSV␣file..." << endl;
160     string filename = "data_clean.csv";
161     vector<vector<float>> data = read_csv(filename);
162
163     //print_first_five_rows(data);
164
165
166     //--------------------
167     // GENERAL SETUP
168     //--------------------
169
```

```
    int num_features = data[0].size() - 1; // number of features (last
        column is the target)
    cout << "Number of Features: " << num_features << endl;


    // --------------------
    // DATA SPLITTING
    // --------------------

    vector<vector<float>> train_data, test_data;
    float test_size = 0.2f; // 20% of the data for testing
    split_data(data, train_data, test_data, test_size);

    vector<vector<float>> train_features, test_features;
    vector<float> train_targets, test_targets;
    split_features_targets(train_data, train_features, train_targets);
    split_features_targets(test_data, test_features, test_targets);


    // --------------------
    // SETUP
    // --------------------

    auto start_gpu = high_resolution_clock::now();



    int num_train_samples = train_features.size();
    int num_test_samples = test_features.size();
    cout << "Train: " << num_train_samples << " | Test: " <<
        num_test_samples << endl;

    float* h_train_features = new float[num_train_samples *  num_features];
    float* h_train_targets = new float[num_train_samples];
    float* h_test_features = new float[num_test_samples * num_features];
    float* h_test_targets = new float[num_test_samples];
    float* h_test_predictions = new float[num_test_samples];

    // copy the features and targets from the dataset to the h_features and
         h_targets pointers
    for (int i = 0; i < num_train_samples; ++i) {
        for (int j = 0; j < num_features; ++j){
            h_train_features[i * num_features + j] = train_features[i][j];
        }
        h_train_targets[i] = train_targets[i];
    }

    for (int i = 0; i < num_test_samples; ++i) {
        for (int j = 0; j < num_features; ++j)
            h_test_features[i * num_features + j] = test_features[i][j];
    }

    for (int i = 0; i < num_test_samples; ++i) {
        h_test_targets[i] = test_targets[i];
    }


    float* d_train_features, *d_test_features, *d_train_targets, *
        d_test_predictions;

    cudaMalloc((void**)&d_train_features, num_train_samples * num_features
```

```
        * sizeof(float));
    cudaMalloc((void**)&d_test_features, num_test_samples * num_features *
        sizeof(float));
    cudaMalloc((void**)&d_train_targets, num_train_samples * sizeof(float))
        ;
    cudaMalloc((void**)&d_test_predictions, num_test_samples * sizeof(float
        ));

    cudaMemcpy(d_train_features, h_train_features, num_train_samples *
        num_features * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_test_features, h_test_features, num_test_samples *
        num_features * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_train_targets, h_train_targets, num_train_samples * sizeof
        (float), cudaMemcpyHostToDevice);

    // -------------------
    // KNN ALGORITHM
    // -------------------

    int k = 10; // number of neighbors
    cout << "Number␣of␣neighbours:␣" << k << endl;
    int blockSize = 256;
    int numBlocks = (num_train_samples + blockSize - 1) / blockSize;

    knn_kernel<<<numBlocks, blockSize>>>(d_train_features, d_test_features,
         d_train_targets, d_test_predictions, num_train_samples,
        num_test_samples, num_features, k);
    cudaDeviceSynchronize();

    cudaMemcpy(h_test_predictions, d_test_predictions, num_test_samples *
        sizeof(float), cudaMemcpyDeviceToHost);


    auto stop_gpu = high_resolution_clock::now();
    auto duration_gpu = duration_cast<milliseconds>(stop_gpu - start_gpu).
        count();
    cout << "GPU␣KNN␣Execution␣Time:␣" << duration_gpu << "␣ms" << endl;


    float accuracy = calculate_accuracy(h_test_predictions, h_test_targets,
         num_test_samples);
    cout << "GPU␣KNN␣Accuracy:␣" << accuracy * 100.0f << "%" << endl;


    cudaFree(d_train_features);
    cudaFree(d_test_features);
    cudaFree(d_train_targets);
    cudaFree(d_test_predictions);


    delete[] h_train_features;
    delete[] h_train_targets;
    delete[] h_test_predictions;
    delete[] h_test_features;
    delete[] h_test_targets;

    cudaDeviceReset();

    return 0;
}
```